方向一：密码学算法可视化教学与 AI 辅导平台（校园/实训向）

真实痛点：密码学课程中，AES 的轮函数、RSA 的模幂运算、Diffie-Hellman 的交换过程，仅靠教材文字和静态图很难建立直觉。现有的一些可视化项目（如 AlgoVault）已经证明“逐步动画 + 交互参数调节”能显著降低理解门槛。但多数工具缺少一个能针对学生当前操作给出解释的“辅导层”。

Web **功能设计**：选择一个或几个核心算法（建议 AES 或 RSA，不要贪多），实现**逐步可视化**：用户输入明文/密钥，页面逐步展示状态矩阵变化、轮密钥生成、模幂计算等中间结果。在此基础上，加入**一个“解释当前步骤”的对话面板**——学生可以选中任意一步，问“这一步在做什么”“为什么这里要异或”，系统返回针对该步骤的解释。

码道结合点：这个方向与码道智能体的结合非常自然。码道可以作为“辅导层”的引擎，在开发阶段辅助生成算法步骤的可视化逻辑；在运行阶段，将学生选中的步骤上下文提供给智能体，生成针对性的解释。你不需要自己训练模型，而是把码道的代码理解与自然语言生成能力嵌入教学交互中。

可行性与工作量：中等。可视化部分工作量集中在“把算法拆解为可展示的步骤”，建议只选一个算法做深，而不是铺开做多个算法的基础演示。

方向二：面向小团队的本地优先端到端加密文件暂存工具

真实痛点：在校园项目组、小型工作室中，经常需要在成员之间临时传递包含敏感信息的文件（成绩表、合同草稿、配置凭证）。直接用微信/QQ 传输缺乏端到端加密保证，而搭建一套完整的密钥管理体系又超出小团队的维护能力。

Web **功能设计**：用户在浏览器中**本地选择文件**，使用 Web Crypto API 在**客户端完成加密**后再上传密文，服务器只存储密文和必要的元数据。接收方通过分享链接获取密文，在浏览器中本地解密。密钥通过链接片段（URL fragment）传递，**永远不发送到服务器**。可以设置密文的一次性下载或过期时间。核心设计原则是：服务器即使被入侵，攻击者也拿不到明文。

码道结合点：码道可以辅助生成 Web Crypto API 的加密封装、文件分块处理逻辑和前端解密流程。密码学相关的代码容易写出“看起来能用但存在侧信道或错误处理缺陷”的实现，码道在生成代码的同时可以辅助你审查关键路径（如 IV 管理、异常处理是否泄露信息）。

可行性与工作量：中等。核心流程清晰（本地加密 → 上传密文 → 下载 → 本地解密），不需要服务端维护密钥。需要注意的边界：**密钥不能出现在服务器日志中**，分享链接的 fragment 部分不会发送到服务器，这一点要在设计和文档中明确说明。

方向 2 的说明：

一、“用户在浏览器中本地选择文件”是什么意思

指的是：用户通过网页上的文件选择控件（<input type="file"> 或拖拽区域），从**自己电脑的文件系统**中选取文件。关键在于**文件内容进入的是浏览器内存，而不是先上传到服务器**。

前端 JavaScript 拿到的是一个 File 对象，可以读取为 ArrayBuffer，然后直接调用浏览器的 **Web Crypto API** 在本地完成加密。也就是说：

- **明文文件从未离开用户的浏览器**；
- 上传到服务器的只有加密后的密文；
- 服务器没有能力解密，因为它从头到尾没有接触过密钥和明文。

对比一下另一种常见做法：“用户上传文件 → 服务器收到明文 → 服务器加密存储”。那种方式下，服务器在加密前已经看到了明文，一旦服务器被入侵或运维人员有权限，明文就是暴露的。本地加密的意义正是消除这个暴露窗口。

二、URL fragment 是什么，为什么服务器看不到它

一个完整的 URL 长这样：

text

```
https://example.com/share/abc123#k=9f2c8e...
└──────────── 路径部分 ────────────┘ └─ fragment ─┘
```

后面的部分叫 **fragment（片段标识符）**。根据 HTTP 规范，浏览器向服务器发起请求时，**只发送 # 之前的部分**，fragment 由浏览器自己保留和处理，不会出现在 HTTP 请求行里，也不会出现在服务器的访问日志中。

所以当接收方打开 https://example.com/share/abc123#k=9f2c8e... 时：

1. 浏览器向服务器请求 /share/abc123，**不带** #k=...；
2. 服务器返回一个页面（HTML + JS）；
3. 页面里的 JavaScript 通过 location.hash 读取到 #k=9f2c8e...，在本地拿到密钥。

服务器全程只知道自己返回了一个页面，**从未收到过密钥**。

三、完整流程：密钥不经过服务器，怎么解密

发送方（加密 + 上传）

1. 用户在浏览器中选择文件，JS 读取文件内容为 ArrayBuffer。
2. 用 crypto.subtle.generateKey 生成一个随机 AES-GCM 密钥（也可以生成随机字节作为密钥材料）。
3. 用 Web Crypto API 在本地加密文件，得到密文。
4. 把**密文 + 必要的元数据**（文件名、IV、过期时间等）上传到服务器。
5. 服务器存储密文，返回一个分享 ID（如 abc123）。
6. 前端把密钥导出并编码（如 base64url），拼成分享链接：https://站点/share/abc123#k=<密钥>。

接收方（下载 + 解密）

1. 接收方打开分享链接。浏览器请求 /share/abc123（**fragment 不发送**），服务器返回页面。
2. 页面 JS 从 location.hash 中取出密钥。
3. JS 向服务器请求密文（按 ID 下载），服务器返回密文。
4. **在浏览器内存中**用密钥解密密文，得到明文，触发下载或展示。

整个过程中，密钥只存在于两个地方：**发送方浏览器的内存**和 **URL 的 fragment 部分**。服务器只有密文和 ID，没有密钥，所以即使数据库泄露，攻击者也拿不到明文。

四、这个设计的边界与注意事项

这个方案的安全性建立在“fragment 不发送给服务器”这一浏览器行为上，但它并不是万能的，设计文档和答辩时需要说清楚：

1. **链接本身就是凭证**。谁拿到完整链接（含 fragment），谁就能解密。所以分享链接要像密码一样对待，建议配合**一次性下载**或**过期时间**来降低链接被转发/截获后的风险。
2. fragment **可能残留在浏览器历史、剪贴板、聊天记录中**。这是使用体验和安全的权衡，可以在页面上提示用户“链接含密钥，请通过可信渠道传递”。
3. **前端代码的完整性很重要**。如果页面 JS 被篡改（比如被注入恶意脚本），密钥就可能在解密前被窃取。所以站点应启用 HTTPS、配置 CSP，并在文档中说明这一信任假设。
4. **密钥编码方式**：通常把密钥的原始字节用 base64url 编码放进 fragment，避免 +、/、= 在 URL 中引发转义问题。

一句话总结：**文件在浏览器里加密，密文上传；密钥放在 URL 的 # 后面，浏览器不会把它发给服务器；接收方打开链接后，浏览器自己从 # 后面取出密钥，在本地解密。**服务器在整个链路中只是一个“密文中转站”，没有解密能力。

方向三：校园场景下的“轻量凭证签发与验证”工具

真实痛点：校园中有大量“证明某件事”的需求——社团活动参与证明、实验室设备使用资格、项目组内部的任务完成确认。传统的做法是发一张截图或一份文档，接收方无法便捷地验证其真实性，也容易被伪造。

Web **功能设计**：签发方（社团负责人、实验室管理员）在平台上填写凭证内容（如“张三已完成设备安全培训”），系统生成一个**带有数字签名的凭证**（可以是一个链接或一段可复制的文本）。验证方打开链接，系统验证签名并展示凭证内容和有效性，同时显示签发方身份。关键设计：**验证方不需要拥有签发方的公钥文件**，公钥由平台托管，但签名验证在服务端可审计地完成，或者使用可验证凭证（VC）的简化模型。

码道结合点：码道可以辅助设计签名方案的选择（Ed25519 通常比 RSA 更适合这种轻量场景）、凭证的数据结构和验证接口。你还可以用码道生成一个“凭证模板配置”模块，让签发方自定义字段而不需要改代码。

可行性与工作量：中低。核心是签名/验签逻辑和凭证的序列化格式，Web 端交互相对简单。适合作为“密码学在真实业务中的最小闭环”来呈现。

方向三说明：

先设定一个具体场景

假设你做了一个“校园活动参与证明”平台。**计算机学院青年志愿者协会**（简称青协）组织了一次志愿活动，活动结束后，青协要给参与者**张三**发一张“参与证明”。后来张三去申请奖学金，**奖学金评审老师**需要确认这张证明是真的、不是张三自己 P 的。

这个场景里有四个角色：

角色	在这个场景里是谁
签发方	青协（有资格签发证明的组织）
持有者	张三（拿到证明的人）
验证方	奖学金评审老师（需要确认证明真实性的人）
平台	你开发的这个 Web 系统

逐句拆解那句话

1.“签发方”是指什么

签发方就是有资格对某件事做出声明、并对声明负责的主体。在数字签名体系里，签发方持有一对密钥：**私钥**用来对凭证签名，**公钥**用来让别人验证签名。

在校园场景里，签发方可以是：社团、实验室管理员、学院教务、项目组负责人。关键点是：**签发方的身份需要被平台认证过**——平台要确认“这个账号确实代表青协”，否则谁都能注册一个“青协”去签发假证明。这就是为什么签发方通常需要平台管理员审核或绑定学校统一身份认证。

2.“验证方不需要拥有签发方的公钥文件”是什么意思

数字签名验证的数学前提是：**验证方必须拿到签发方的公钥**。没有公钥，就无法验证签名。

传统做法是：验证方自己去下载、保存、管理签发方的公钥文件（类似“公钥基础设施 PKI”里的证书分发）。但这在校园场景里不现实——评审老师不可能为了验证一张社团证明，去下载青协的公钥文件、还要确认这个公钥文件本身没被替换。

所以“验证方不需要拥有公钥文件”的意思是：**验证方不直接持有公钥，而是把验证这件事委托给平台，由平台用托管的公钥完成验签，再把结果告诉验证方。**

3.“公钥由平台托管”，平台指谁

平台就是你开发的这个 Web 系统（比如“校园凭证签发与验证平台”）。签发方在注册时，把自己的公钥上传到平台（或者由平台生成密钥对、私钥交给签发方保管、公钥留在平台），平台负责：

- 记录“这个公钥属于青协”；
- 在验证时，根据凭证里的签发方标识，找到对应公钥；
- 用这个公钥执行验签。

平台在这里扮演的是**可信第三方**的角色。它的可信性来自：平台本身由学校或可信组织运营，签发方身份经过审核，公钥与身份的绑定关系由平台维护。

4.“签名验证在服务端可审计地完成”是什么意思

这句话包含两层：

第一层：验证发生在服务端，不是验证方本地。 验证方（评审老师）打开一个链接或上传凭证，请求发到平台服务器，服务器用托管的公钥验签，返回“有效/无效”以及凭证内容。验证方不需要懂密码学，也不需要装任何软件。

第二层：“可审计”是指验签过程有记录、可追溯、不是黑箱。 具体可以包括：

- 平台记录每一次验证请求：谁在什么时间验证了哪张凭证、结果是什么；
- 验签逻辑是确定性的、可复查的，不是“平台说有效就有效”；
- 凭证本身携带签名和签发方标识，第三方（比如学校信息中心）可以用同样的公钥独立复验，不完全依赖平台的单点结论；
- 如果发生争议（比如张三说证明是真的、老师说是假的），可以调取验证日志和凭证原始数据来追溯。

“可审计”的反面是“平台内部偷偷改结果”——所以设计上要保证：**平台不能伪造验证结果而不留痕迹**，凭证的签名数据要能被独立复验。

5.“可验证凭证（VC）的简化模型”是什么

VC（Verifiable Credential，**可验证凭证**）是 W3C 提出的一个标准模型，核心思想是：凭证由签发方签名，持有者可以出示，验证方可以验证，而不需要每次都回源到签发方查询。

它的完整模型涉及去中心化标识符（DID）、可验证表述（Verifiable Presentation）、选择性披露等，对参赛项目来说太重了。**“简化模型”**就是保留核心思想、砍掉复杂部分：

- 凭证是一个结构化数据（JSON），包含：签发方标识、持有者标识、声明内容、签发时间、签名值；
- 签发方用私钥对凭证内容签名；
- 验证时，平台根据签发方标识找到公钥，验签，确认内容未被篡改、签发方身份可信；
- 不做完整的 DID 体系，签发方身份由平台账号体系承载；
- 不做选择性披露，凭证内容整体出示。

把整个流程串起来

签发阶段：

1. 青协登录平台（身份已审核），填写“张三参与 XX 志愿活动”。
2. 平台用青协托管的私钥（或青协本地私钥）对凭证内容签名，生成凭证（JSON 或链接）。
3. 张三拿到凭证链接。

验证阶段：

1. 评审老师打开凭证链接，请求发到平台。
2. 平台解析凭证，根据签发方标识找到青协的公钥，验签。
3. 平台返回：凭证内容 + 验证结果（有效/无效）+ 签发方信息。
4. 平台记录这次验证的审计日志。

关键点：评审老师全程没有接触公钥文件，他信任的是“平台托管的公钥与青协身份的绑定关系”。而“可审计”保证了平台不能随意篡改验证结论而不留痕迹。

设计文档里需要写清楚的两件事

1. **信任模型**：平台是可信第三方，签发方身份经过审核，公钥与身份的绑定由平台维护。验证方信任平台，但凭证签名数据本身可被独立复验。
2. **私钥归属**：如果私钥由平台托管，平台就拥有签发能力，需要严格的权限控制和操作日志；如果私钥由签发方本地保管（比如青协自己持有），平台只托管公钥，安全性更高但使用门槛也更高。参赛项目可以根据团队能力选择，但要在文档里明确说明并解释取舍。

方向四：面向非技术用户的“密码学词义”

方向四：面向非技术用户的“密码学闯关”科普应用（趣味化方向）

真实痛点：网络安全的公众科普往往停留在“不要用弱密码”的口号层面。密码学的核心思想（为什么公钥可以公开、哈希为什么不可逆、数字签名为什么能防篡改）用文字解释枯燥，但用**交互式谜题**来呈现，学习效果会截然不同。已有学生团队将密码学原理做成网页小游戏并获得实际落地签约，说明这个方向在竞赛和真实场景中都有可行性。

Web 功能设计：设计 3—5 个递进式关卡，每个关卡对应一个密码学概念。例如：第一关“凯撒密码破译”，让用户通过频率分析还原密文；第二关“哈希碰撞演示”，展示两个不同输入产生相同前缀哈希的困难性（通过可视化搜索过程）；第三关“中间人攻击模拟”，让用户在 Diffie-Hellman 交换中扮演攻击者，直观感受“没有身份认证的密钥交换为什么危险”。每关结束后给出概念总结。面向中学生或非技术专业学生。

码道结合点：码道可以辅助生成关卡逻辑、前端交互和提示系统。更有价值的结合是：当用户卡关时，码道生成的**渐进式提示**可以根据用户当前状态给出方向性引导，而不是直接给答案——这比静态提示更接近“有辅导能力的科普工具”。

可行性与工作量：中低。关卡数量控制在 3—5 个，每个关卡的核心交互逻辑不复杂。密码学概念的准确性需要仔细校对，避免为了趣味性而传递错误的技术结论。

注意事项

上面四个课题仅作参考，各位同学可按自己实际情况选上面课题或其他课题（但要符合大方向：密码学、web 应用程序，完整 Web 网站项目成果包含全部源代码、部署配置说明、项目操作使用文档； 开发过程证明材料（基于华为云 CodeArts 的代码托管、版本管理、流水线发布等过程记录截图或链接），具体见报名参赛页面）